

的输入，然后解码得到第二轮的输出。接着，将第二轮的输出和输入拼接，作为第三轮的输入，以此类推。每次将本轮预测到的词拼接到本轮的输入上，输入给语言模型，完成下一轮预测。在循环迭代的“自回归”过程中，我们不断生成新的词，这些词便构成了一段文本。

但上述“自回归”过程存在着两个问题：(1) **错误级联放大**，选用模型自己生成的词作为输入可能会有错误，这样的错误循环输入，将会不断的放大错误，导致模型不能很好拟合训练集；(2) **串行计算效率低**，因为下一个要预测的词依赖上一次的预测，每次预测之间是串行的，难以进行并行加速。为了解决上述两个问题，“**Teacher Forcing**” [21] 在语言模型预训练过程中被广泛应用。在 Teacher Forcing 中，每轮都仅将输出结果与“标准答案”（Ground Truth）进行拼接作为下一轮的输入。在图1.4所示的例子中，第二轮循环中，我们用“长颈鹿脖子”来预测下一个词“长”，而非选用 o_1 中概率最高的词“吃”或者其他可能输出的词。

但是，Teacher Forcing 的训练方式将导致**曝光偏差**（Exposure Bias）的问题。曝光偏差是指 Teacher Forcing 训练模型的过程和模型在推理过程存在差异。Teacher Forcing 在训练中，模型将依赖于“标准答案”进行下一次的预测，但是在推理预测中，模型“自回归”的产生文本，没有“标准答案”可参考。所以模型在训练过程中和推理过程中存在偏差，可能推理效果较差。为解决曝光偏差的问题，Bengio 等人提出了针对 RNN 提出了 Scheduled Sampling 方法 [2]。其在 Teacher Forcing 的训练过程中循序渐进的使用一小部分模型自己生成的词代替“标准答案”，在训练过程中对推理中无“标准答案”的情况进行预演。

由于 RNN 模型循环迭代的本质，其不易进行并行计算，导致其在输入序列较长时，训练较慢。下节将对容易并行的基于 Transformer 的语言模型进行介绍。

1.3 基于 Transformer 的语言模型

Transformer 是一类基于注意力机制（Attention）的**模块化**构建的神经网络结构。给定一个序列，Transformer 将一定数量的历史状态和当前状态同时输入，然后进行加权相加。对历史状态和当前状态进行“**通盘考虑**”，然后对未来状态进行预测。基于 Transformer 的语言模型，以词序列作为输入，基于一定长度的上文和当前词来预测下一个词出现的概率。本节将先对 Transformer 的基本原理进行介绍，然后讲解如何利用 Transformer 构建语言模型。

1.3.1 Transformer

Transformer 是由两种模块组合构建的**模块化网络结构**。两种模块分别为：(1) 注意力（Attention）模块；(2) 全连接前馈（Fully-connected Feedforward）模块。其中，自注意力模块由自注意力层（Self-Attention Layer）、残差连接（Residual Connections）和层正则化（Layer Normalization）组成。全连接前馈模块由全连接前馈层，残差连接和层正则化组成。两个模块的结构示意图如图1.5所示。以下详细介绍每个层的原理及作用。

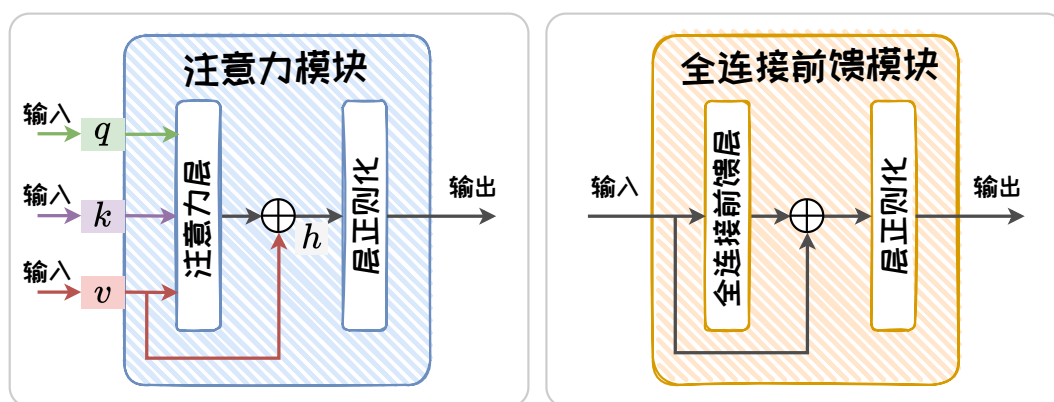


图 1.5: 注意力模块与全连接前馈模块。

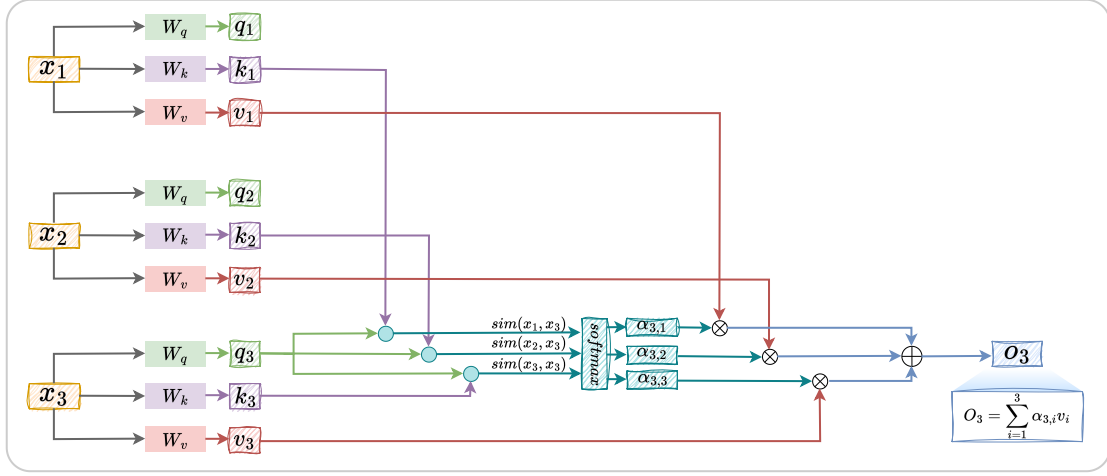


图 1.6: 注意力机制示意图。

1. 注意力层 (Attention Layer)

注意力层采用加权平均的思想将前文信息叠加到当前状态上。Transformer 的注意力层将输入编码为 query, key, value 三部分, 即将输入 $\{x_1, x_2, \dots, x_t\}$ 编码为 $\{(q_1, k_1, v_1), (q_2, k_2, v_2), \dots, (q_t, k_t, v_t)\}$ 。其中, query 和 key 用于计算自注意力的权重 α , value 是对输入的编码。具体的,

$$Attention(x_t) = \sum_{i=1}^t \alpha_{t,i} v_i. \quad (1.31)$$

其中,

$$\alpha_{t,i} = softmax(sim(x_t, x_i)) = \frac{sim(q_t, k_i)}{\sum_{i=1}^t sim(q_t, k_i)}. \quad (1.32)$$

其中, $sim(\cdot, \cdot)$ 用于度量两个输入之间的相关程度, softmax 函数用于对此相关程度进行归一化。此外,

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i, \quad (1.33)$$

其中, W_q, W_k, W_v 分别为 query, key, value 编码器的参数。以包含三个元素的输入 $\{x_1, x_2, x_3\}$ 为例, Transformer 自注意力的实现图 1.6 所示。

2. 全连接前馈层 (Fully-connected Feedforward Layer)

全连接前馈层占据了 Transformer 近三分之二的参数，掌管着 Transformer 模型的记忆。其可以看作是一种 Key-Value 模式的记忆存储管理模块 [7]。全连接前馈层包含两层，两层之间由 ReLU 作为激活函数。设全连接前馈层的输入为 v ，全连接前馈层可由下式表示：

$$FFN(v) = \max(0, W_1 v + b_1) W_2 + b_2. \quad (1.34)$$

其中， W_1 和 W_2 分别为第一层和第二层的权重参数， b_1 和 b_2 分别为第一层和第二层的偏置参数。其中第一层的可看作神经记忆中的 key，而第二层可看作 value。

3. 层正则化 (Layer Normalization)

层正则化用以加速神经网络训练过程并取得更好的泛化性能 [1]。设输入到层正则化层的向量为 $v = \{v_i\}_{i=1}^n$ 。层正则化层将在 v 的每一维度 v_i 上都进行层正则化操作。具体地，层正则化操作可以表示为下列公式：

$$LN(v_i) = \frac{\alpha(v_i - \mu)}{\delta} + \beta. \quad (1.35)$$

其中， α 和 β 为可学习参数。 μ 和 δ 分别是隐藏状态的均值和方差，可由下列公式分别计算。

$$\mu = \frac{1}{n} \sum_{i=1}^n v_i, \quad \delta = \sqrt{\frac{1}{n} \sum_{i=1}^n (v_i - \mu)^2}. \quad (1.36)$$

4. 残差连接 (Residual Connections)

引入残差连接可以有效解决梯度消失问题。在基本的 Transformer 编码模块中包含两个残差连接。第一个残差连接是将自注意力层的输入由一条旁路叠加到自注意力层的输出上，然后输入给层正则化。第二个残差连接是将全连接前馈层的输入由一条旁路引到全连接前馈层的输出上，然后输入给层正则化。

上述将层正则化置于残差连接之后的网络结构被称为 Post-LN Transformer。与之相对的，还有一种将层正则化置于残差连接之前的网络结构，称之为 Pre-LN

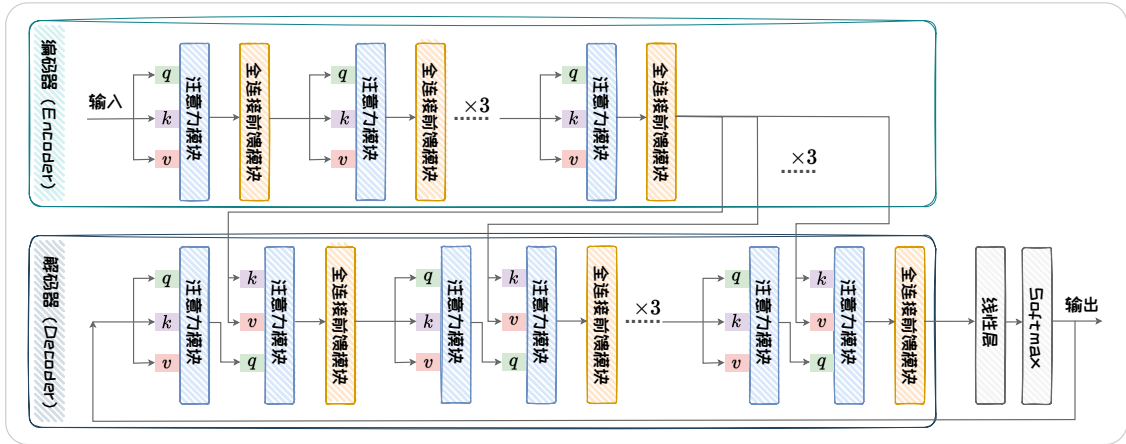


图 1.7: Transformer 结构示意图。

Transformers。对比两者, Post-LN Transformer 应对表征坍塌 (Representation Collapse) 的能力更强, 但处理梯度消失略弱。而 Pre-LN Transformers 可以更好的应对梯度消失, 但处理表征坍塌的能力略弱。具体分析可参考文献 [7, 22]。

原始的 Transformer 采用 Encoder-Decoder 架构, 其包含 Encoder 和 Decoder 两部分。这两部分都是由自注意力模块和全连接前馈模块重复连接构建而成。其整体结构如图1.7所示。其中, Encoder 部分由六个级联的 encoder layer 组成, 每个 encoder layer 包含一个注意力模块和一个全连接前馈模块。其中的注意力模块为**自注意力模块** (query, key, value 的输入是相同的)。Decoder 部分由六个级联的 decoder layer 组成, 每个 decoder layer 包含两个注意力模块和一个全连接前馈模块。其中, 第一个注意力模块为**自注意力模块**, 第二个注意力模块为**交叉注意力模块** (query, key, value 的输入不同)。Decoder 中第一个 decoder layer 的自注意力模块的输入为模型的输出。其后的 decoder layer 的自注意力模块的输入为上一个 decoder layer 的输出。Decoder 交叉注意力模块的输入分别是自注意力模块的输出 (query) 和最后一个 encoder layer 的输出 (key, value)。

Transformer 的 Encoder 部分和 Decoder 部分都可以单独用于构造语言模型, 分别对应 Encoder-Only 模型和 Decoder-Only 模型。Encoder-Only 模型和 Decoder-Only 模型的具体结构将在第二章中进行详细介绍。

1.3.2 基于 Transformer 的语言模型

在 Transformer 的基础上, 可以设计多种预训练任务来训练语言模型。例如, 我们可以基于 Transformer 的 Encoder 部分, 结合“掩词补全”等任务来训练 Encoder-Only 语言模型, 如 BERT [5]; 我们可以同时应用 Transformer 的 Encoder 和 Decoder 部分, 结合“截断补全”、“顺序恢复”等多个有监督和自监督任务来训练 Encoder-Decoder 语言模型, 如 T5 [18]; 我们可以同时应用 Transformer 的 Decoder 部分, 利用“下一词预测”任务来训练 Decoder-Only 语言模型, 如 GPT-3 [3]。这些语言模型将在第二章中进行详细介绍。下面将以下一词预测任务为例, 简单介绍训练 Transformer 语言模型的流程。

对词序列 $\{w_1, w_2, w_3, \dots, w_N\}$, 基于 Transformer 的语言模型根据 $\{w_1, w_2, \dots, w_i\}$ 预测下一个词 w_{i+1} 出现的概率。在基于 Transformer 的语言模型中, 输出为一个向量, 其中每一维代表着词典中对应词的概率。设词典 D 中共有 $|D|$ 个词 $\{\hat{w}_1, \hat{w}_2, \dots, \hat{w}_{|D|}\}$ 。基于 Transformer 的语言模型的输出可表示为 $o_i = \{o_i[\hat{w}_d]\}_{d=1}^{|D|}$, 其中, $o_i[\hat{w}_d]$ 表示词典中的词 $\hat{w}_d$ 出现的概率。因此, 对 Transformer 的语言模型对词序列 $\{w_1, w_2, w_3, \dots, w_N\}$ 整体出现的概率的预测为:

$$P(w_{1:N}) = \prod_{i=1}^{N-1} P(w_{i+1}|w_{1:i}) = \prod_{i=1}^N o_i[w_{i+1}] \quad (1.37)$$

与训练 RNN 语言模型相同, Transformer 语言模型也常用如下交叉熵函数作为损失函数。

$$l_{CE}(o_i) = - \sum_{d=1}^{|D|} I(\hat{w}_d = w_{i+1}) \log o_i[w_{i+1}] = - \log o_i[w_{i+1}]. \quad (1.38)$$

其中, $I(\cdot)$ 为指示函数, 当 $\hat{w}_d = w_{i+1}$ 时等于 1, 当 $\hat{w}_d \neq w_{i+1}$ 时等于 0。

设训练集为 S , Transformer 语言模型的损失可以构造为:

$$L(S, W) = \frac{1}{N|S|} \sum_{s=1}^{|S|} \sum_{i=1}^N l_{CE}(o_{i,s}) \quad (1.39)$$